Python Advanced Course
Exercise 02
SoSe 2025

Chris Geron
Sophia Sirtl, Linus Rodríguez Gómez, Miguel Cortina Zapp, Jonas Reif

Ausgabe: semptember 9, 2025

Note: This sheet contains more exercises than can probably be completed during the tutorial. Take a look at the exercises and start with the ones that appeal to you first.
Feel free to extend or modify the given exercises with any functionality you like.

In some exercises you will find references to already prepared programs. You can find and download these from this Sciebo share[1] in the folder of the corresponding day.

Setting up a Development Environment

(1) If this has not already been done on a previous day, set up a virtual environment and a development environment.
You can find more about this on Sheet 1.

(2) Extend your `venv` with the packages `fastapi` and `requests`.

Wiki Race
The Wikipedia game has the goal of finding the fastest path from a starting article to a predefined target article by clicking only on links within the articles.
You are now to implement this game for the console. Starting with the article on `Ball_python`, all possible links should be displayed on the console.

For this, you can use the pre-made URL of the Wikipedia API[2].
With `input()`, the desired next article from the displayed selection can be chosen.
The program should then load and display the new links of the corresponding page.
The game is won once the article `Python_(programming_language)` is reached.

Note: The Wikipedia API requires you to specify a user agent. You don't need to fake a legitimate browser, it's just important to say what you are. For example:

```
headers = {
 "User-Agent": "LectureExampleBot/1.0 (contact: your-real@email.com)"
}
```

[1] https://uni-bonn.sciebo.de/s/f3yJSCBKiDLHzHS
[2] https://wikipedia.org/w/api.php?action=query&prop=links&pllimit=max&format=json&titles=SEARCH-TERM

Packaging a Program
The goal of this exercise is to package a program of your choice into a program executable from the command line. The task is broken down into very small steps, so it may look like more than it actually is.
The task assumes that you use the sample project as starting point. You can also build your projectstructure from scratch. Then you can skip steps 1 to 4.

(1) Download the `PyPa sample project`[3].

(2) Extract it to a location of your choice.

(3) Give the package a meaningful name.

(4) Move it out of the `src/` folder into your projects root.

- Remove the now empty `src/` folder
- If you want to: make sure to adjust all properties in the `pyproject.toml` accordingly
- (Having the modules in `src/` has some sideeffects, you can just avoid by not having that subfolder.)

(5) Insert a program – e.g. your Wiki Race implementation – into `simple.py` (feel free to rename this file too!).

The important part here is that all functionalities are wrapped in functions (or classes), and that the program can be started by calling a function.

(6) Import your start function into `__init__.py` and call it in `main()`.

(7) Adjust the `pyproject.toml` so that the metadata corresponds to your project. (Simply delete optional content that is not needed.)

In particular, adjust the name of your command line program under `[project.scripts]`.

(8) Open a terminal, ensure you are in the root directory of your project and that your `venv` is activated[4].

(9) Install your package with `python3 -m pip install -e .`

(10) Call the command line program you created.

Typehinting
The goal of this exercise is to familiarize you with more advanced concepts of type hinting.

(1) Add meaningful type hints to the code in `typehinting.py`.

Note: Think about `type aliases`.

(2) Create functions and objects that match the type hints given later in the file.

[3] `https://github.com/pypa/sampleproject/archive/refs/heads/main.zip`
[4] `https://docs.python.org/3/library/venv.html#how-venvs-work`
[4] The `venv` does not have to be in the same directory. You can also switch directories after activating the `venv`.

Mutable Keyword Argument Defaults

- Define a function that has an empty `list` as default value for a keyword argument.
- Add a function body that does at least append one element to the list and print it afterwards.
- Call the function mutliple times in your script and observe the behavior.
- Finally, change the function in a way that it creates a new empty list every time the function is called, while still maintaining the keyword argument structure, with a reasonable default.

Pydantic

The goal is to model bands and their members as data classes.

(1) Model the given UML diagram. You may extend the model with any functionalities and helper functions.

Tips for proceeding:

- Generally make sure to create a good string representation for your models early on.
- Start by modeling `BandMember`.
- Then create the rough design of the `Band` model.
- Finally, add `json` serializability.

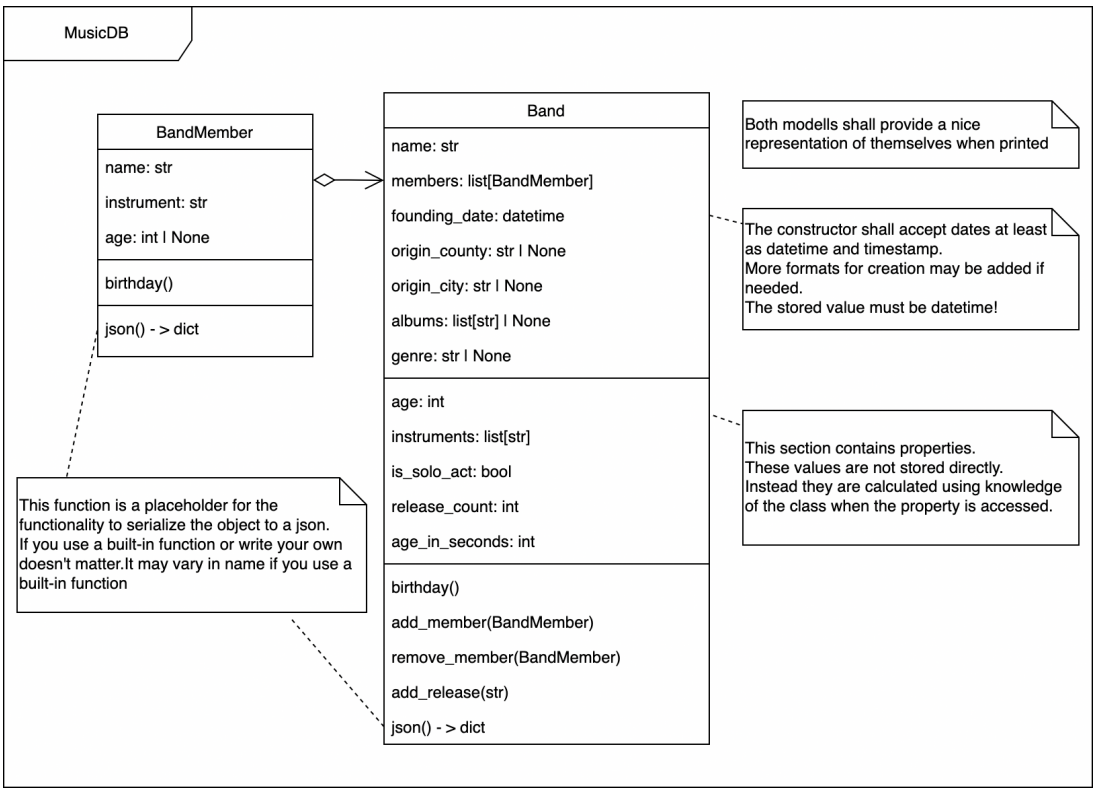

Abbildung 1: The UML diagram to be modeled

FastAPI
The goal of this exercise is to give you ideas for a possible API. Two systems of roughly equal complexity are proposed below.
Again, you are free to choose whether to use `Pydantic` or `dataclass` for your data models.
Notify:

- A route to which messages (e.g. text) can be sent.
- The server stores these messages internally.
- Another route allows retrieval of all stored messages.

  Possible approach:

  - Consider what additional context the message should include.

    Should a sender or the timestamp of receipt be stored as well? How about a unique identifier – keyword `uuid`?
  - Model a data class that matches your ideas.
  - Define a route that can receive a message and provides the corresponding parameters.
  - Store the messages internally.
  - Add another route that allows querying the messages.

    Should there perhaps be an option to query only the first n messages?
- Possible extensions:
  - A route to query whether any data is available
  - Multiple differently named channels
  - An overview of which channels exist (and whether data is available).

URL Shortener:

- A route that allows registering a URL along with the desired short link.
- A route that, when the short link is called, returns the original link.

  Possible approach:

  - Consider what additional context a shortened URL should contain.

    Should perhaps the creator or the timestamp of creation be stored?
  - Model a data class that matches your ideas.
  - Define a route that can receive a message and provides the corresponding parameters.
  - Store the messages internally.

    How do you ensure that each short link is only available once?
  - Add another route that allows calling the shortened links.
- Possible extensions:
  - An overview of all shortened links.

    Should links be able to be explicitly registered as invisible?

- A route to remove links.

  How can we ensure that only the creator can delete the link? What kind of unique identifier could they get?
- Should the name of the shortened link be optional?

  How could a random link be generated?
- Tricky: Would you like to allow storing images?

HTTP Header Manipulation with `requests`
The goal of this exercise is to deepen your understanding of working with HTTP headers in web requests. The following task requires you to explicitly set headers such as `User-Agent` and `Referer` in order to access an API.

API Query with Custom Headers:

- Send a GET request to an API at the URL: `https://supersecretstuff.miguelcz.com/validate-request`.
- Set the following HTTP headers:
  - `User-Agent`: Use a legitimate browser agent such as 'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:130.0) Gecko/20100101 SecretStuffApp/42.0'.
  - `Referer`: Set the value to 'https://supersecretstuff.miguelcz.com'.
- Check the status code of the response. If the code is not 200, output a meaningful error message.
- If the status code is 200, parse the JSON response and display the content in the console.

  Possible approach:

  - Use the `requests` module in Python to send the GET request.
  - Set the required HTTP headers in a dictionary and pass them to the request.
  - Use the `response.json()` method to parse the response.